Buffer : Object

Client-side representation of a buffer on a server

Source: Buffer.sc

See also: ExampleFiles

Description

A Buffer object is a client-side abstraction for a server-side buffer. (SuperCollider's server-client architecture is a common source of confusion when working with Buffer objects, so please see Client vs Server.)

A buffer is most often used to hold sampled audio, such as a soundfile loaded into memory, but can be used to hold other types of data as well. Technically speaking, a buffer on the server is a globally available, multichannel array of 32-bit floating-point numbers. It also has an associated sample rate, represented in Hertz as a 64-bit float.

The Buffer class encapsulates a number of common tasks, OSC messages, and capabilities related to server-side buffers – see the examples lower down this document for many examples of using Buffers for sound playback and recording.

Buffers are commonly used with PlayBuf, RecordBuf, DiskIn, DiskOut, BufWr, BufRd, and other UGens. (See their individual help files for more examples.) Buffers can be freed or altered even while being accessed. See Server Architecture for some technical details.

Buffer objects should not be created or modified within a SynthDef. If this is needed, see LocalBuf.

Buffer Numbers and Allocation

Although the number of buffers on a server is set at the time it is booted, memory must still be allocated within the server app before they can hold values. (At boot time all buffers have a size of 0.)

Server-side buffers are identified by number, starting from 0. When using Buffer objects, buffer numbers are automatically allocated from the Server's bufferAllocator. When you call `.free` on a Buffer object it will release the buffer's memory on the server, and free the buffer number for future reallocation. See ServerOptions for details on setting the number of available buffers.

Normally you should not need to supply a buffer number (see Buffer: *alloc). You should only do so if you are sure you know what you are doing. Similarly, in normal use you should not need to access the buffer number, since instances of Buffer can be used directly as UGen inputs or Synth args.

Multichannel Buffers

Multichannel buffers interleave their data. Thus the actual number of available values when requesting or setting values by index using methods such as `set, setn, get, getn`, etc., is equal to `numFrames * numChannels`. Indices start at 0 and go up to `(numFrames * numChannels) - 1`. In a two channel buffer for instance, index 0 will be the first value of the first channel, index 1 will be the first value of the second channel, index 2 will be the second value of the first channel, and so on.

In some cases it is simpler to use multiple single channel buffers instead of a single multichannel one.

Completion Messages and Action Functions

Many buffer operations (such as reading and writing files) are asynchronous, meaning that they will take an arbitrary amount of time to complete. Asynchronous commands are passed to a background thread on the server so as not to steal CPU time from the audio synthesis thread. Since they can last an arbitrary amount of time it is convenient to be able to specify something else that can be done immediately on completion. The ability to do this is implemented in two ways in Buffer's various methods: completion messages and action functions.

A completion message is a second OSC command which is included in the message which is sent to the server. (See Node Messaging for a discussion of OSC messages.) The server will execute this immediately upon completing the first command. An action function is a Function which will be evaluated when the client receives the appropriate reply from the

Action functions are passed the Buffer object as an argument when they are evaluated.

With Buffer methods that take a completion message, it is also possible to pass in a function that returns an OSC message. As in action functions this will be passed the Buffer as an argument. It is important to understand however that this function will be evaluated after the Buffer object has been created (so that its bufnum and other details are accessible), but before the corresponding message is sent to the server.

Bundling

Many of the methods below have two versions: a regular one which sends its corresponding message to the server immediately, and one which returns the message in an Array so that it can be added to a bundle. It is also possible to capture the messages generated by the regular methods using Server's automated bundling capabilities. See Server and Bundled Server Messages for more details.

Server message data types

The server expects `numFrames`, `numChannels` and get/set index arguments to be integers. As a courtesy, the Buffer methods to construct server messages (method names ending in 'Msg') convert floating-point numbers to integers for these arguments. Users constructing messages manually should take care with these data types.

Class Methods

Creation with Immediate Memory Allocation

```
Buffer.alloc(server, numFrames, numChannels: 1, completionMessage, bufnum,
       sampleRate)
```

Create and return a Buffer and immediately allocate the required memory on the server. The buffer's values will be initialised to 0.0.

Arguments:

server	The server on which to allocate the buffer. The default is the default Server.
numFrames	The number of frames to allocate. Actual memory use will correspond to numFrames * numChannels. (A floating-point value is truncated to integer.)
numChannels	The number of channels for the Buffer. The default is 1. (A floating-point value is truncated to integer.)
completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.
sampleRate	The sample rate of the buffer. The default is the sample rate of the server. 0 or nil will reset to the Server's sample rate.

Discussion:

```
// Allocate 8 second stereo buffer
s.boot;
b = Buffer.alloc(s, s.sampleRate * 8.0, 2);
b.free;
```

Allocates a range of consecutively-numbered buffers, for use with UGens like VOsc and VOsc3 that require a contiguous block of buffers, and returns an array of corresponding Buffer objects.

Arguments:

numBufs	The number of consecutively indexed buffers to allocate.
server	The server on which to allocate the buffers. The default is the default Server.
numFrames	The number of frames to allocate in each buffer. Actual memory use will correspond to numFrames * numChannels. (A floating-point value is truncated to integer.)
numChannels	The number of channels for each buffer. The default is 1. (A floating-point value is truncated to integer.)
completionMessage	A valid OSC message or a Function which will return one. A Function will be passed each Buffer and its index in the array as arguments when evaluated.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.
sampleRate	The sample rate of the buffer. The default is the sample rate of the server.

Discussion:

N.B. You must treat the array of Buffers as a group. Freeing them individually or reusing them can result in allocation errors. You should free all Buffers in the array at the same time by iterating over it with do.

```
s.boot;
// allocate an array of Buffers and fill them with different harmonics
(
b = Buffer.allocConsecutive(8, s, 4096, 1, { |buf, i|
    buf.sine1Msg((1 .. ((i + 1) * 6)).reciprocal) // completion messages
});

a = { VOsc.ar(SinOsc.kr(0.5, 0).range(b.first.bufnum + 0.1, b.last.bufnum -
0.1),
    [440, 441], 0, 0.2) }.play;
)
a.free;

// iterate over the array and free it
b.do(_.free);
```

Buffer.**read**(server, path, startFrame: 0, numFrames: -1, action, bufnum)

Allocate a buffer and immediately read a soundfile into it. This method sends a query message as a completion message so that the Buffer's instance variables will be updated automatically.

Arguments:

server	The server on which to allocate the buffer.
path	A String representing the path of the soundfile to be read.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file.
numFrames	The number of frames to read. The default is -1, which will read the whole file.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.

choose the next available bufnum.

Discussion:

N.B. You cannot rely on the buffer's instance variables being instantly updated, as there is a small amount of latency involved. action will be evaluated upon receipt of the reply to the query, so use this in cases where access to instance variables is needed.

```
// Read a sound file from disk into a buffer
s.boot;
// ExampleFiles helps locate audio files used in examples
p = ExampleFiles.child;
// peek at the path to see location and the format of a path
p.postln;
// read the file
b = Buffer.read(s, p);

// now play it
(
x = SynthDef(\help_Buffer, { |out = 0, bufnum|
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum))
    )
}).play(s, [\bufnum, b]);
)
x.free; b.free;

// with an action function
// note that the vars are not immediately up-to-date
(
b = Buffer.read(s, p, action: { |buffer|
    ("After update:" + buffer.numFrames).postln;
    x = { PlayBuf.ar(1, buffer, BufRateScale.kr(buffer)) }.play;
});
("Before update:" + b.numFrames).postln;
)
x.free; b.free;
```

Buffer.**readChannel**(server, path, startFrame: 0, numFrames: -1, channels, action, bufnum)

As *read above, but takes an Array of channel indices to read in, allowing one to read only the selected channels.

Arguments:

server	The server on which to allocate the buffer.
path	A String representing the path of the soundfile to be read.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file.
numFrames	The number of frames to read. The default is -1, which will read the whole file.
channels	An Array of channels to be read from the soundfile. Indices start from zero. These will be read in the order provided.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.

```
s.boot;
// first a standard read so we can see what's in the file
b = Buffer.read(s, ExampleFiles.sinedPink);
// ExampleFiles.sinedPink contains SinOsc on left, PinkNoise on right
b.plot;
b.free;

// Now just the sine
b = Buffer.readChannel(s, ExampleFiles.sinedPink, channels: [0]);
b.plot;
b.free;

// Now just the pink noise
b = Buffer.readChannel(s, ExampleFiles.sinedPink, channels: [1]);
b.plot;
b.free;

// Now reverse channel order
b = Buffer.readChannel(s, ExampleFiles.sinedPink, channels: [1, 0]);
b.plot;
b.free;
```

Buffer.**readNoUpdate**(server, path, startFrame: 0, numFrames: -1, bufnum, completionMessage)

As *read above, but without the automatic update of instance variables. Call `updateInfo` (see below) to update the vars.

Arguments:

server	The server on which to allocate the buffer.
path	A String representing the path of the soundfile to be read.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file.
numFrames	The number of frames to read. The default is -1, which will read the whole file.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.
completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

Discussion:

```
// with a completion message
s.boot;
(
SynthDef(\help_Buffer, { |out = 0, bufnum|
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum))
    )
}).add;

y = Synth.basicNew(\help_Buffer); // not sent yet
b = Buffer.readNoUpdate(s, ExampleFiles.child,
    completionMessage: { |buffer|
        // synth add its s_new msg to follow
        // after the buffer read completes
        y.newMsg(s, [\bufnum, buffer], \addToTail)
```

```
// note vars not accurate
b.numFrames; // nil
b.updateInfo;
b.numFrames; // 188893
// when done...
y.free;
b.free;
```

Buffer.**cueSoundFile**(server, path, startFrame: 0, numChannels: 2, bufferSize: 32768, completionMessage)

Allocate a buffer and preload a soundfile for streaming in using DiskIn.

Arguments:

server	The server on which to allocate the buffer.
path	A String representing the path of the soundfile to be read.
startFrame	The frame of the soundfile that DiskIn will start playing at.
numChannels	The number of channels in the soundfile.
bufferSize	This must be a multiple of (2 * the server's block size). 32768 is the default and is suitable for most cases.
completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

Discussion:

```
s.boot;
(
SynthDef(\help_Buffer_cue, { |out = 0, bufnum|
    Out.ar(out,
        DiskIn.ar(1, bufnum)
    )
}).add;
)

(
s.makeBundle(nil, {
    b = Buffer.cueSoundFile(s, ExampleFiles.child, 0, 1);
    y = Synth(\help_Buffer_cue, [\bufnum, b], s);
});
)
b.free; y.free;
```

Buffer.**loadCollection**(server, collection, numChannels: 1, action)

Load a large collection into a buffer on the server. Returns a Buffer object.

Arguments:

server	The server on which to create the buffer.
collection	A subclass of Collection (i.e. an Array) containing only floats and integers. Multi-dimensional arrays will not work.
numChannels	The number of channels that the buffer should have. Note that buffers interleave multichannel data. You are responsible for providing an interleaved collection if needed. Multi-dimensional arrays will

action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.

Discussion:

This is accomplished through writing the collection to a SoundFile and loading it from there. For this reason this method will only work with a server on your local machine. For a remote server use `sendCollection`, below. The file is automatically deleted after loading. This allows for larger collections than setn, below, and is in general the safest way to get a large collection into a buffer. The sample rate of the buffer will be the sample rate of the server on which it is created.

```
s.boot;
(
a = FloatArray.fill(44100 * 5.0, { 1.0.rand2 }); // 5 seconds of noise
b = Buffer.loadCollection(s, a);
)

// test it
b.get(20000, { |msg| (msg == a[20000]).postln });

// play it
x = { PlayBuf.ar(1, b, BufRateScale.kr(b), loop: 0) * 0.5 }.play;
b.free; x.free;

// interleave a multi-dimensional array
(
l = Signal.sineFill(16384, Array.fill(200, { 0 }).add(1));
r = Array.fill(16384, { 1.0.rand2 });
m = [Array.newFrom(l), r]; // a multi-dimensional array
m = m.lace(32768); // interleave the two collections
b = Buffer.loadCollection(s, m, 2, { |buf|
    x = { PlayBuf.ar(2, buf, BufRateScale.kr(buf), loop: 1) * 0.5 }.play;
});
)
b.plot;
x.free; b.free;
```

Buffer.**sendCollection**(server, collection, numChannels: 1, wait: -1, action)

Stream a large collection into a buffer on the server using multiple setn messages. Returns a Buffer object.

Arguments:

server	The server on which to create the buffer.
collection	A subclass of Collection (i.e. an Array) containing only floats and integers. Multi-dimensional arrays will not work.
numChannels	The number of channels that the buffer should have. Note that buffers interleave multichannel data. You are responsible for providing an interleaved collection if needed. Multi-dimensional arrays will not work. See the example in *loadCollection above, to see how to do this.
wait	An optional wait time between sending setn messages. In a high traffic situation it may be safer to set this to something above zero, which is the default.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.

Discussion:

This allows for larger collections than setn, below. This is not as safe as *loadCollection above, but will work with servers on remote machines. The sample rate of the buffer will be the sample rate of the server on which it is created.

```
a = Array.fill(2000000, { rrand(0.0, 1.0) }); // a LARGE collection
b = Buffer.sendCollection(s, a, 1, 0, { |buf| "finished".postln });
)
b.get(1999999, { |msg| (msg == a[1999999]).postln });
b.free;
```

Buffer.**loadDialog**(server, startFrame: 0, numFrames, action, bufnum)

As *read above, but gives you a load dialog window to browse for a file.

Arguments:

server	The server on which to allocate the buffer.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file.
numFrames	The number of frames to read. The default is -1, which will read the whole file.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.

Discussion:

```
s.boot;
(
b = Buffer.loadDialog(s, action: { |buffer|
    x = { PlayBuf.ar(buffer.numChannels, buffer, BufRateScale.kr(buffer))
}.play;
});
)
x.free; b.free;
```

Buffer.**loadChannelDialog**(server, startFrame: 0, numFrames, channels, action, bufnum)

As *readChannel above, but gives you a load dialog window to browse for a file.

Arguments:

server	The server on which to allocate the buffer.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file.
numFrames	The number of frames to read. The default is -1, which will read the whole file.
channels	An Array of channels to be read from the soundfile. Indices start from zero. These will be read in the order provided. If absent or an empty array all channels will be read from soundfile in order.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.

Discussion:

```
s.boot;
```

```
    x = { PlayBuf.ar(buffer.numChannels, buffer, BufRateScale.kr(buffer)) }.play;
});
)
x.free; b.free;
```

Creation without Immediate Memory Allocation

`Buffer.new(server, numFrames, numChannels, bufnum, sampleRate)`

Create and return a new Buffer object, without immediately allocating the corresponding memory on the server. This combined with 'message' methods can be flexible with bundles.

Arguments:

server	The server on which to allocate the buffer. The default is the default Server.
numFrames	The number of frames to allocate. Actual memory use will correspond to numFrames * numChannels.
numChannels	The number of channels for the Buffer. The default is 1.
bufnum	An explicitly specified buffer number. Supplying a number bypasses the server's buffer allocator, which can lead to conflicts. The best practice is to leave this unset and let the buffer number allocator choose the next available bufnum.
sampleRate	The sample rate of the buffer. The default is the sample rate of the server.

Discussion:

```
s.boot;
b = Buffer.new(s, 44100 * 8.0, 2);
c = Buffer.new(s, 44100 * 4.0, 2);
b.query; // numFrames = 0
s.sendBundle(nil, b.allocMsg, c.allocMsg); // sent both at the same time
b.query; // now it's right
c.query;
b.free; c.free;
```

Cached Buffers

To assist with automatic updates of buffer information (see `updateInfo` and `read`), buffer objects are cached in a collection associated with the Server object hosting the buffers. Freeing a buffer removes it from the cache; quitting the server clears all the cached buffers. (This also occurs if the server crashes unexpectedly.)

You may access cached buffers using the following methods.

It may be simpler to access them through the server object:

```
myServer.cachedBufferAt(bufnum)
myServer.cachedBuffersDo(func)

b = Buffer.alloc(s, 2048, 1);
Buffer.cachedBufferAt(s, 0);    // assuming b has bufnum 0
s.cachedBufferAt(0);            // same result
s.cachedBuffersDo({ |buf| buf.postln });
```

`Buffer.cachedBufferAt(server, bufnum)`

`Buffer.`**`cachedBuffersDo`**`(server, func)`

Iterate over all cached buffers. The iteration is not in any order, but will touch all buffers.

Inherited class methods

7 methods from Object ► show

Undocumented class methods

`Buffer.`**`freeAll`**`(server)`

Instance Methods

Variables

The following variables have getter methods.

`.server`

Returns the Buffer's Server object.

`.bufnum`

Returns the buffer number of the corresponding server-side buffer. In normal use you should not need to access this value, since instances of Buffer can be used directly as UGen inputs or Synth args.

Discussion:

```
s.boot;
b = Buffer.alloc(s, 44100 * 8.0, 2);
b.bufnum.postln;
b.free;
```

`.numFrames`

`.numFrames` `= value`

Returns the number of sample frames in the corresponding server-side buffer. Note that multichannel buffers interleave their samples, so when dealing with indices in methods like get and getn, the actual number of available values is numFrames * numChannels.

`.numChannels`

`.numChannels` `= value`

Returns the number of channels in the corresponding server-side buffer.

`.sampleRate`

`.sampleRate` `= value`

The `sampleRate` setter method only changes the language-side instance variable. It does not resample the buffer or change its sample rate on the server. To change the server-side buffer sample rate, use `setSampleRate`.

NOTE: Setting the `sampleRate` instance variable affects the returned -duration (which returns `numFrames / sampleRate`).

WARNING: You can use -query to inspect the buffer's sample rate on the server, but this will also *update the* `sampleRate` *instance variable*, overwriting its value which you may have previously set.

.setSampleRate(value)

Sets the sample rate of the buffer on the server side, and updates its value on the language side. This does not resample the audio.

Arguments:

value	The new desired sample rate. 0 or nil will set to the Server's sample rate.

.duration

Returns `this.numFrames / this.sampleRate` (language-side instance variables).

.path

.path = value

Returns a string containing the path of a soundfile that has been loaded into the corresponding server-side buffer.

Explicit allocation

These methods allocate the necessary memory on the server for a Buffer previously created with *new.

.alloc(completionMessage)

.allocMsg(completionMessage)

Arguments:

completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

.allocRead(argpath, startFrame: 0, numFrames: -1, completionMessage)

.allocReadMsg(argpath, startFrame: 0, numFrames: -1, completionMessage)

Read a soundfile into a buffer on the server for a Buffer previously created with *new. Note that this will not autoupdate instance variables. Call `updateInfo` in order to do this.

Arguments:

argpath	A String representing the path of the soundfile to be read.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file. (A floating-point value is truncated to integer.)

completionMessage A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

Discussion:

```
s.boot;
b = Buffer.new(s);
b.allocRead(ExampleFiles.child);
x = { PlayBuf.ar(1, b, BufRateScale.kr(b), loop: 1) * 0.5 }.play;
x.free; b.free;
```

.allocReadChannel(argpath, startFrame: 0, numFrames: -1, channels, completionMessage)

.allocReadChannelMsg(argpath, startFrame: 0, numFrames: -1, channels, completionMessage)

As -allocRead above, but allows you to specify which channels to read.

Arguments:

argpath A String representing the path of the soundfile to be read.

startFrame The first frame of the soundfile to read. The default is 0, which is the beginning of the file. (A floating-point value is truncated to integer.)

numFrames The number of frames to read. The default is -1, which will read the whole file. (A floating-point value is truncated to integer.)

channels An Array of channels to be read from the soundfile. Indices start from zero. These will be read in the order provided. If absent or an empty array all channels will be read from soundfile in order.

completionMessage A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

Discussion:

```
s.boot;
b = Buffer.new(s);
// read only the first channel (a Sine wave) of a stereo file
b.allocReadChannel(ExampleFiles.sinedPink, channels: [0]);
x = { PlayBuf.ar(1, b, BufRateScale.kr(b), loop: 1) * 0.5 }.play;
x.free; b.free;
```

Other methods

.read(argpath, fileStartFrame: 0, numFrames: -1, bufStartFrame: 0, leaveOpen: false, action)

Read a soundfile into an already allocated buffer.

Arguments:

argpath A String representing the path of the soundfile to be read.

fileStartFrame The first frame of the soundfile to read. The default is 0, which is the beginning of the file. (A floating-point value is truncated to integer.)

bufStartFrame	The index of the frame in the buffer at which to start reading. The default is 0, which is the beginning of the buffer. (A floating-point value is truncated to integer.)
leaveOpen	A boolean indicating whether or not the Buffer should be left 'open'. For use with DiskIn you will want this to be true, as the buffer will be used for streaming the soundfile in from disk. (For this the buffer must have been allocated with a multiple of (2 * synth block size). A common number is 32768 frames. cueSoundFile below, provides a simpler way of doing this.) The default is false which is the correct value for all other cases.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.
(completionMessage)	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

.**readMsg**(argpath, fileStartFrame: 0, numFrames: -1, bufStartFrame: 0, leaveOpen: false, completionMessage)

construct the message for a read command. args are like those for read, except that last arg is completionMessage.

Discussion:

Note that if the number of frames in the file is greater than the number of frames in the buffer, it will be truncated. Note that readMsg will not auto-update instance variables. Call updateInfo in order to do this.

.**readChannel**(argpath, fileStartFrame: 0, numFrames: -1, bufStartFrame: 0, leaveOpen: false, channels, action)

As -read above, but allows you to specify which channels to read.

Arguments:

argpath	A String representing the path of the soundfile to be read.
fileStartFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file. (A floating-point value is truncated to integer.)
numFrames	The number of frames to read. The default is -1, which will read the whole file. (A floating-point value is truncated to integer.)
bufStartFrame	The index of the frame in the buffer at which to start reading. The default is 0, which is the beginning of the buffer. (A floating-point value is truncated to integer.)
leaveOpen	A boolean indicating whether or not the Buffer should be left 'open'. For use with DiskIn you will want this to be true, as the buffer will be used for streaming the soundfile in from disk. (For this the buffer must have been allocated with a multiple of (2 * synth block size). A common number is 32768 frames. cueSoundFile below, provides a simpler way of doing this.) The default is false which is the correct value for all other cases.
channels	An Array of channels to be read from the soundfile. Indices start from zero. These will be read in the order provided. The number of channels requested must match this Buffer's numChannels.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.
(completionMessage)	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

.**readChannelMsg**(argpath, fileStartFrame: 0, numFrames: -1, bufStartFrame: 0, leaveOpen: false, channels, completionMessage)

.cueSoundFile(path, startFrame, completionMessage)

.cueSoundFileMsg(path, startFrame: 0, completionMessage)

A convenience method to cue a soundfile into the buffer for use with a DiskIn. The buffer must have been allocated with a multiple of (2 * the server's block size) frames. A common size is 32768 frames.

Arguments:

path	A String representing the path of the soundfile to be read.
startFrame	The first frame of the soundfile to read. The default is 0, which is the beginning of the file. (A floating-point value is truncated to integer.)
completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

Discussion:

```
s.boot;



// create with cueSoundFile class method
b = Buffer.cueSoundFile(s, ExampleFiles.apollo11, 0, 1);
x = { DiskIn.ar(1, b) }.play;
b.close;    // must call close in between cueing
// now use like named instance method, but different arguments
b.cueSoundFile(ExampleFiles.apollo11);
// have to do all this to clean up properly!
x.free; b.close; b.free;
```

.write(path, headerFormat: "aiff", sampleFormat: "int24", numFrames: -1, startFrame: 0, leaveOpen: false, completionMessage)

.writeMsg(path, headerFormat: "aiff", sampleFormat: "int24", numFrames: -1, startFrame: 0, leaveOpen: false, completionMessage)

Write the contents of the buffer to a file. See SoundFile for information on valid values for headerFormat and sampleFormat.

Arguments:

path	A String representing the path of the soundfile to be written. If no path is given, Buffer writes into the default recording directory with a generic name.
headerFormat	A String.
sampleFormat	A String.
numFrames	The number of frames to write. The default is -1, which will write the whole buffer. (A floating-point value is truncated to integer.)
startFrame	The index of the frame in the buffer from which to start writing. The default is 0, which is the beginning of the buffer. (A floating-point value is truncated to integer.)
leaveOpen	A boolean indicating whether or not the Buffer should be left 'open'. For use with DiskOut you will want this to be true. The default is false which is the correct value for all other cases.
completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

.freeMsg(completionMessage)

Release the buffer's memory on the server and return the bufferID back to the server's buffer number allocator for future reuse.

Arguments:

completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

.zero(completionMessage)

.zeroMsg(completionMessage)

Sets all values in this buffer to 0.0.

Arguments:

completionMessage	A valid OSC message or a Function which will return one. A Function will be passed this Buffer as an argument when evaluated.

.set(index, float ... morePairs)

.setMsg(index, float ... morePairs)

Set the value in the buffer at index to be equal to float. Additional pairs of indices and floats may be included in the same message. (Floating-point values for index are truncated to integer.)

Discussion:

Note that multichannel buffers interleave their sample data, therefore the actual number of available values is equal to `numFrames * numChannels`. Indices start at 0.

```
s.boot;
b = Buffer.alloc(s, 4, 2);
b.set(0, 0.2, 1, 0.3, 7, 0.4); // set the values at indices 0, 1, and 7.
b.getn(0, 8, { |msg| msg.postln });
b.free;
```

.setn(... args)

.setnMsg(... args)

Set a contiguous range of values in the buffer starting at the index startAt to be equal to the Array of floats or integers, values. The number of values set corresponds to the size of values. Additional pairs of starting indices and arrays of values may be included in the same message. (Floating-point values for index are truncated to integer.)

Discussion:

Note that multichannel buffers interleave their sample data, therefore the actual number of available values is equal to `numFrames * numChannels`. You are responsible for interleaving the data in values if needed. Multi-dimensional arrays will not work. Indices start at 0.

N.B. The maximum number of values that you can set with a single setn message is 1633 when the server is using UDP as its communication protocol. Use -loadCollection and -sendCollection to set larger ranges of values.

```
s.boot;
b = Buffer.alloc(s, 16);
b.setn(0, Array.fill(16, { rrand(0, 1) }));
b.getn(0, b.numFrames, { |msg| msg.postln });
b.setn(0, [1, 2, 3], 4, [1, 2, 3]);
```

.loadCollection(collection, startFrame: 0, action)

Load a large collection into this buffer. This is accomplished through writing the collection to a SoundFile and loading it from there. For this reason this method will only work with a server on your local machine. For a remote server use sendCollection, below. The file is automatically deleted after loading.

Arguments:

collection	A subclass of Collection (i.e. an Array) containing only floats and integers. Multi-dimensional arrays will not work.
startFrame	The index of the frame at which to start loading the collection. The default is 0, which is the start of the buffer.
action	A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.

Discussion:

This allows for larger collections than setn, above, and is in general the safest way to get a large collection into a buffer. The sample rate of the buffer will be the sample rate of the server on which it was created. The number of channels and frames will have been determined when the buffer was allocated. You are responsible for making sure that the size of collection is not greater than numFrames, and for interleaving any data if needed.

```
s.boot;
(
v = Signal.sineFill(128, 1.0 / [1, 2, 3, 4, 5, 6]);
b = Buffer.alloc(s, 128);
)
(
b.loadCollection(v, action: { |buf|
    x = { PlayBuf.ar(buf.numChannels, buf, BufRateScale.kr(buf), loop: 1)
        * 0.2 }.play;
});
)
x.free; b.free;

// interleave a multi-dimensional array
(
l = Signal.sineFill(16384, Array.fill(200, { 0 }).add(1));
r = Array.fill(16384, { 1.0.rand2 });
m = [Array.newFrom(l), r]; // a multi-dimensional array
m = m.lace(32768); // interleave the two collections
b = Buffer.alloc(s, 16384, 2);
)
(
b.loadCollection(m, 0, { |buf|
    x = { PlayBuf.ar(2, buf, BufRateScale.kr(buf), loop: 1) * 0.5 }.play;
});
)
b.plot;
x.free; b.free;
```

.sendCollection(collection, startFrame: 0, wait: -1, action)

Stream a large collection into this buffer using multiple setn messages.

Arguments:

startFrame The index of the frame at which to start streaming in the collection. The default is 0, which is the start of the buffer.

wait An optional wait time between sending setn messages. In a high traffic situation it may be safer to set this to something above zero, which is the default.

action A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.

Discussion:

This allows for larger collections than setn. This is not as safe as loadCollection, above, but will work with servers on remote machines. The sample rate of the buffer will be the sample rate of the server on which it is created.

```
s.boot;
(
a = Array.fill(2000000, { rrand(0.0, 1.0) });
b = Buffer.alloc(s, 2000000);
)
b = b.sendCollection(a, action: { |buf| "finished".postln });
b.get(1999999, { |msg| (msg == a[1999999]).postln });
b.free;
```

.get(index, action)

.getMsg(index)

Send a message requesting the value in the buffer at index. action is a Function which will be passed the value as an argument and evaluated when a reply is received. (Floating-point values for index are truncated to integer.)

Discussion:

```
s.boot;
b = Buffer.alloc(s, 16);
b.setn(0, Array.fill(16, { rrand(0.0, 1.0) }));
b.get(0, { |msg| msg.postln });
b.free;
```

.getn(index, count, action)

.getnMsg(index, count, action)

Send a message requesting the a contiguous range of values of size count starting from index. action is a Function which will be passed the values in an Array as an argument and evaluated when a reply is received. (Floating-point values for index and count are truncated to integer.)

Discussion:

N.B. The maximum number of values that you can get with a single getn message is 1633 when the server is using UDP as its communication protocol. Use -loadToFloatArray and -getToFloatArray to get larger ranges of values.

.loadToFloatArray(index: 0, count: -1, action)

Write the buffer to a file and then load it into a FloatArray.

Arguments:

index The index in the buffer to begin writing from. The default is 0.

action A Function which will be passed the resulting FloatArray as an argument and evaluated when loading is finished.

Discussion:

This is safer than getToFloatArray but only works with a server on your local machine. In general this is the safest way to get a large range of values from a server buffer into the client app.

```
s.boot;
b = Buffer.read(s, ExampleFiles.child);
// same as Buffer.plot
b.loadToFloatArray(action: { |array| { array.plot }.defer; "done".postln });
b.free;
```

.getToFloatArray(index: 0, count: -1, wait: 0.01, timeout: 3, action)

Stream the buffer to the client using a series of getn messages and put the results into a FloatArray.

Arguments:

index The index in the buffer to begin writing from. The default is 0. (A floating-point value is truncated to integer.)

count The number of values to fetch. The default is -1, which gets from index until the end of the buffer. (A floating-point value is truncated to integer.)

wait The amount of time in seconds to wait between sending getn messages. Longer times are safer. The default is 0.01 seconds which seems reliable under normal circumstances. A setting of 0 is not recommended.

timeout The amount of time in seconds after which to post a warning if all replies have not yet been received. the default is 3.

action A Function which will be passed the resulting FloatArray as an argument and evaluated when all replies have been received.

Discussion:

This is more risky than loadToFloatArray but does works with servers on remote machines. In high traffic situations it is possible for data to be lost. If this method has not received all its replies by timeout it will post a warning saying that the method has failed. In general use loadToFloatArray instead wherever possible.

```
s.boot;
b = Buffer.read(s, ExampleFiles.child);
// like Buffer.plot
b.getToFloatArray(wait: 0.01, action: { |array { array.plot }.defer;
"done".postln });
b.free;
```

.normalize(newmax: 1, asWavetable: false)

.normalizeMsg(newmax: 1, asWavetable: false)

Normalizes the buffer so that the peak absolute value is newmax (which defaults to 1). If your buffer is in Wavetable format then set the asWavetable argument to true.

.fill(startAt, numFrames, value ... more)

.fillMsg(startAt, numFrames, value ... more)

.copyData(buf, dstStartAt: 0, srcStartAt: 0, numSamples: -1)

.copyMsg(buf, dstStartAt: 0, srcStartAt: 0, numSamples: -1)

Starting at the index srcStartAt, copy numSamples samples from this to the destination buffer buf starting at dstStartAt. If numSamples is negative, the maximum number of samples possible is copied. The default is start from 0 in the source and copy the maximum number possible starting at 0 in the destination. (Floating-point values for srcStartAt, numSamples and dstStartAt are truncated to integer.)

Discussion:

Note: This method used to be called copy, but this conflicts with the method for object-copying. Therefore Buffer:copy is now intended to create a copy of the client-side Buffer object. Its use for copying buffer data on the server is deprecated. If you see a deprecation warning, the data will still be copied on the server and your code will still work, but you should update your code for the new method name.

```
s.boot;
(
SynthDef(\help_Buffer_copy, { |out = 0, buf|
    Line.ar(0, 0, dur: BufDur.kr(buf), doneAction: Done.freeSelf);
    Out.ar(out, PlayBuf.ar(1, buf));
}).add;
)

(
b = Buffer.read(s, ExampleFiles.child);
c = Buffer.alloc(s, 120000);
)

Synth(\help_Buffer_copy, [\buf, b]);

// copy the whole buffer
b.copyData(c);
Synth(\help_Buffer_copy, [\buf, c]);

// copy some samples
c.zero;
b.copyData(c, numSamples: 4410);
Synth(\help_Buffer_copy, [\buf, c]);

// buffer "compositing"
c.zero;
b.copyData(c, numSamples: 4410);
b.copyData(c, dstStartAt: 4410, numSamples: 15500);
Synth(\help_Buffer_copy, [\buf, c]);

b.free;
c.free;
```

.close(completionMessage)

.closeMsg(completionMessage)

After using a Buffer with a DiskOut or DiskIn, it is necessary to close the soundfile. Failure to do so can cause problems.

Arguments:

.plot(name, bounds, minval, maxval, separately: false, parent)

From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/GUI/PlusGUI/Math/PlotView.sc

Plot the contents of the Buffer in a GUI window.

Arguments:

name The name of the resulting window.

bounds An instance of Rect determining the size of the resulting view.

minval the minimum value in the plot

maxval the maximum value in the plot

separately a boolean whether to use separate display ranges or a single shared range.

parent Either a Window or View may be passed in - then the plot is embedded. Otherwise a new Window is created.

```
(
s.waitForBoot {
    var w = Window("parent");
    var buf = Buffer.read(s, ExampleFiles.sinedPink);
    s.sync;
    buf.plot(parent: w);
    w.front
}
)
```

Discussion:

```
s.boot;
b = Buffer.read(s, ExampleFiles.child);
b.plot;
b.free;
```

.play(loop: false, mul: 1)

Plays the contents of the buffer on the server and returns a corresponding Synth.

Arguments:

loop A Boolean indicating whether or not to loop playback. If false the synth will automatically be freed when done. The default is false.

mul A value by which to scale the output. The default is 1.

Discussion:

```
s.boot;
b = Buffer.read(s, ExampleFiles.child);
b.play; // frees itself
x = b.play(true);
x.free; b.free;
```

.query(action)

order, to the **action** function which, by default, is:

```
action: { |oscAddrPattern, bufnum, numFrames, numChannels, sampleRate|
    postf("bufnum: %\nnumFrames: %\nnumChannels: %\nsampleRate: %\n",
        bufnum, numFrames, numChannels, sampleRate
    );
}
```

which posts the buffer's attributes.

WARNING: Calling `query` will update the `Buffer` instance variables `numFrames`, `numChannels`, and `sampleRate`. This is a side effect of the action of the OSC responders stored in buffer's entry in the `serverCaches`.

Arguments:

action An optional Function to be called with the buffer attributes listed above as arguments, replacing the default **action**.

.**updateInfo**(action)

Sends a b_query message to the server, asking for a description of this buffer. Upon reply this Buffer's instance variables are automatically updated.

Arguments:

action A Function to be evaluated once the file has been read and this Buffer's instance variables have been updated. The function will be passed this Buffer as an argument.

Discussion:

```
s.boot;
b = Buffer.readNoUpdate(s, ExampleFiles.child);
b.numFrames; // shows nil
b.updateInfo({ |buf| buf.numFrames.postln }); // now it's updated
b.free;
```

Buffer Fill Commands

These correspond to the various b_gen OSC Commands, which fill the buffer with values for use. See Server Command Reference for more details.

.**gen**(genCommand, genArgs, normalize: true, asWavetable: true, clearFirst: true)

.**genMsg**(genCommand, genArgs, normalize: true, asWavetable: true, clearFirst: true)

This is a generalized version of the commands below.

Arguments:

genCommand A String indicating the name of the command to use. See Server-Command-Reference for a list of valid command names.

genArgs An Array containing the corresponding arguments to the command.

normalize A Boolean indicating whether or not to normalize the buffer to a peak value of 1.0. The default is true.

asWavetable A Boolean indicating whether or not to write to the buffer in wavetable format so that it can be read by interpolating oscillators. The default is true.

.sine1(amps, normalize: true, asWavetable: true, clearFirst: true)

.sine1Msg(amps, normalize: true, asWavetable: true, clearFirst: true)

Fill this buffer with a series of sine wave harmonics using specified amplitudes.

Arguments:

amps	An Array containing amplitudes for the harmonics. The first float value specifies the amplitude of the first partial, the second float value specifies the amplitude of the second partial, and so on.
normalize	A Boolean indicating whether or not to normalize the buffer to a peak value of 1.0. The default is true.
asWavetable	A Boolean indicating whether or not to write to the buffer in wavetable format so that it can be read by interpolating oscillators. The default is true.
clearFirst	A Boolean indicating whether or not to clear the buffer before writing. The default is true.

Discussion:

```
s.boot;
(
b = Buffer.alloc(s, 512, 1);
b.sine1(1.0 / [1, 2, 3, 4], true, true, true);
x = { Osc.ar(b, 200, 0, 0.5) }.play;
)
x.free; b.free;
```

.sine2(freqs, amps, normalize: true, asWavetable: true, clearFirst: true)

.sine2Msg(freqs, amps, normalize: true, asWavetable: true, clearFirst: true)

Fill this buffer with a series of sine wave partials using specified frequencies and amplitudes.

Arguments:

freqs	An Array containing frequencies (in cycles per buffer) for the partials.
amps	An Array containing amplitudes for the partials. This should contain the same number of items as freqs.
normalize	A Boolean indicating whether or not to normalize the buffer to a peak value of 1.0. The default is true.
asWavetable	A Boolean indicating whether or not to write to the buffer in wavetable format so that it can be read by interpolating oscillators. The default is true.
clearFirst	A Boolean indicating whether or not to clear the buffer before writing. The default is true.

Discussion:

```
s.boot;
(
b = Buffer.alloc(s, 512, 1);
b.sine2([1.0, 3], [1, 0.5]);
x = { Osc.ar(b, 200, 0, 0.5) }.play;
)
x.free; b.free;
```

.sine3Msg(freqs, amps, phases, normalize: true, asWavetable: true, clearFirst: true)

Fill this buffer with a series of sine wave partials using specified frequencies, amplitudes, and initial phases.

Arguments:

freqs	An Array containing frequencies (in cycles per buffer) for the partials.
amps	An Array containing amplitudes for the partials. This should contain the same number of items as freqs.
phases	An Array containing initial phase for the partials (in radians). This should contain the same number of items as freqs.
normalize	A Boolean indicating whether or not to normalize the buffer to a peak value of 1.0. The default is true.
asWavetable	A Boolean indicating whether or not to write to the buffer in wavetable format so that it can be read by interpolating oscillators. The default is true.
clearFirst	A Boolean indicating whether or not to clear the buffer before writing. The default is true.

.cheby(amps, normalize: true, asWavetable: true, clearFirst: true)

.chebyMsg(amps, normalize: true, asWavetable: true, clearFirst: true)

Fill this buffer with a series of Chebyshev polynomials, which can be defined as: `cheby(n) = amplitude * cos(n * acos(x))`. To eliminate a DC offset when used as a waveshaper, the wavetable is offset so that the center value is zero.

Similar functionality can be found in Signal.chebyFill and Wavetable.chebyFill. If you require Chebyshev polynomials that do not include the offset compensation, it is recommended to use one of these.

Arguments:

amps	An Array containing amplitudes for the harmonics. The first float value specifies the amplitude for n = 1, the second float value specifies the amplitude for n = 2, and so on.
normalize	A Boolean indicating whether or not to normalize the buffer to a peak value of 1.0. The default is true.
asWavetable	A Boolean indicating whether or not to write to the buffer in wavetable format so that it can be read by interpolating oscillators. The default is true.
clearFirst	A Boolean indicating whether or not to clear the buffer before writing. The default is true.

Discussion:

NOTE: In previous versions, offsetting (to ensure the center value is zero) was performed incorrectly. This was fixed in version 3.7, so any code that may have relied on the (wrong) behavior may need to be changed. If using a Chebyshev buffer as a waveshaper, the simplest fix is to wrap the Shaper in a LeakDC UGen.

```
s.boot;
b = Buffer.alloc(s, 512, 1, { |buf| buf.chebyMsg([1, 0, 1, 1, 0, 1]) });
(
x = {
    Shaper.ar(
        b,
        SinOsc.ar(300, 0, Line.kr(0, 1, 6)),
        0.5
    )
}.play;
)
```

Inherited instance methods

267 methods from Object ► show

Undocumented instance methods

.asBufWithValues

.asControlInput

.asUGenInput

.cache

.doOnInfo = value

.preparePartConv(buf, fftsize)
From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Audio/PartConv.sc

.queryDone

.queryMsg

.readNoUpdate(argpath, fileStartFrame: 0, numFrames: -1, bufStartFrame: 0, leaveOpen: false, completionMessage)

.setSampleRateMsg(value)

.setnMsgArgs(... args)

.startFrame

.startFrame = value

.streamCollection(collstream, collsize, startFrame: 0, wait: -1, action)

.uncache